

上下文工程：预算、裁剪、缓存与组装

语言策略：code (Java / Python / TypeScript)

TL;DR

1. Agent 效果瓶颈常在 Context Engineering, 不在模型。
2. 每一轮喂给模型的上下文应当是**预算内、按优先级排序、边界清晰**的组装结果。
3. 上下文工程 = 预算 + 选择 + 压缩 + 组装 + 缓存 + 版本化。

1. 上下文预算公式

可用上下文 = 模型窗口 - 保留输出 Token - System Prompt - 工具 Schema
每轮动态预算 = 可用上下文 - 历史对话预算 - 检索预算 - 记忆预算 - 安全余量

规则：保留输出 Token 按任务最大回复长度计算；工具 Schema 只注入本轮白名单工具；安全余量默认留 10%。

2. 内容优先级

优先级	内容	处理策略
P0	任务目标、验收标准、安全规则	永不被裁剪
P1	最近 N 轮观察	保留最近，更早做摘要
P2	工具返回	截断长文本，保留结构化字段
P3	检索片段	只保留 rerank 后的 top-k
P4	历史记录	按相关度注入，低相关不注入

3. 压缩与裁剪

- 工具返回先提取结构化摘要，再决定是否保留原文；
- 历史对话超过预算时，把早期轮次压缩成状态摘要；
- 代码和日志用行号范围截断，不从中部硬切；
- 外部内容统一包裹为数据区，防止指令注入；
- 达到 40% 上下文利用率后，优先裁剪而不是继续塞入。

4. 缓存与版本化

- System Prompt 固定部分放前缀，命中 Prompt Cache；
- 工具 Schema 按 toolset_version 缓存；
- 检索缓存键 = query 归一化 + 知识库版本 + 租户；
- 每次上下文模板变更要 bump prompt_version，并跑评测回归。

5. 组装顺序

System Prompt (版本化)
→ 任务卡与验收标准
→ 本轮白名单工具 Schema
→ 相关记忆 (低相关不注入)
→ 检索片段 (标注来源)
→ 最近观察 (按预算截断)
→ 用户当前输入 (隔离包裹)

6. Java 版上下文组装器

```
public record ContextBudget(int windowTokens, int reservedOutput,
    int systemPrompt,
    int toolSchemas, int safetyReserve) {
    public int available() {
        return windowTokens - reservedOutput - systemPrompt -
            toolSchemas - safetyReserve;
    }
}
```

```
}
}

public record ContextSection(String name, int priority, int tokens,
    String content) {}

public class ContextAssembler {
    public static String assemble(ContextBudget budget, List<
        ContextSection> sections, int maxTokens) {
        int cap = Math.min(maxTokens, budget.available());
        StringBuilder sb = new StringBuilder();
        int used = 0;
        for (ContextSection s : sections.stream()
            .sorted(Comparator.comparingInt(ContextSection::
                priority)).toList()) {
            if (used + s.tokens() > cap && s.priority() > 1)
                continue;
            sb.append("<!-- section:").append(s.name()).append("
            ").append(s.content()).append("
            ");
            used += s.tokens();
        }
        return sb.toString();
    }
}
```

7. 上线检查单

- 每次调用都计算预算，不超模型窗口；
- 工具 Schema 只注入白名单工具；
- 长文本按优先级裁剪，不丢失任务目标；
- 外部数据有数据边界标记；
- Prompt 版本与评测结果绑定；
- 缓存命中率、单任务 Token、上下文利用率有监控。

8. 延伸阅读

- [Agent 记忆系统](#)
- [大模型网关](#)
- [Agent 评测体系](#)